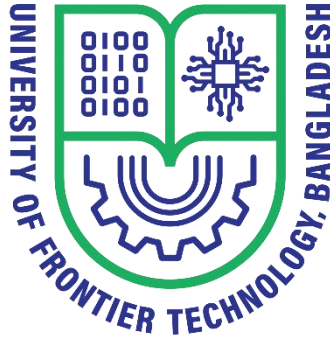




University of Frontier Technology, Bangladesh
Department of Cyber Security Engineering (CySE)

Project Proposal

Project Title : Digital Forensics System

Course Name : Object Oriented Programming Language Sessional

Course Code : PROG 112

Level : 01

Term : 02

Instructor : Md Masud Rana

Lecturer,

Department of Cyber Security Engineering, UFTB

Team Members

Student Name	Student ID	Email
FARIA YASMIN JUI	2404036	2404036@uftb.ac.bd
ATIQR RAHMAN	2404037	2404037@uftb.ac.bd

Digital Forensics System

1. Project Proposal

This project is about building a digital forensic system **using C++**. The Digital Forensic System is a comprehensive platform designed to collect, preserve, analyze, and report digital evidence in a secure and legally admissible manner. The system supports investigations involving cybercrime, data breaches, fraud, and insider threats by automating forensic processes and maintaining evidence integrity through strict chain-of-custody mechanisms.

2. Problem Statement

The main issue with cybercrimes these days is that investigating them needs good tools, but a lot of the ones out there are too complicated or cost a ton. They are not really for students or small setups, and often rely on other software that we have to install separately. So we think there is a gap for a simple tool that runs on its own. It should keep evidence safe, make the process **straightforward** and control how data gets analyzed without risks.

3. Proposed Solution & Objectives

Proposed Solution

Develop a **console-based (or basic GUI) Digital Forensic System in C++** that:

- Collects files as evidence
- Generates hash values for verification
- Analyzes file metadata
- Maintains investigation logs

Objectives

- Build a **standalone forensic tool using C++**
- Ensure **data integrity using hashing**
- Provide **basic forensic analysis features**
- Maintain **chain-of-custody records**
- Practice **OOP concepts in real-world application**

4. Target Audience

- Students (CSE / IT)
- Beginner forensic investigators
- Academic researchers
- Small cybersecurity projects
- Government intelligence units
- Corporate security teams

5. Key Features & Project Scope

Key Features

- File Evidence Collection
- Custom Hashing Method
- Metadata Extraction (size, type, timestamp)
- Keyword Search in files
- Activity Logging System
- Chain-of-Custody Tracking
- Report Generation (TXT file)

Project Scope

- Focus on **local file system analysis**
- No advanced network/mobile forensics
- Works as **offline desktop application**
- Can be extended later with GUI (Qt)

6. Functional & Non-Functional Requirements

Functional Requirements

- Add/Remove evidence files
- Generate hash values
- View file details
- Search content inside files
- Maintain logs of actions
- Export report

Non-Functional Requirements

- Fast file processing
- Secure data handling
- Easy-to-use interface (menu-driven)
- High reliability
- Maintainable code (OOP structure)

7. Technology Stack

Category	Technology / Tool	Purpose in Project
Programming Language	C++	Core application development
Programming Paradigm	Object Oriented Programming(OOP)	Modular and structured design
File Management	fstream	Reading and writing evidence files
Data Structure	vector	Store multiple evidence records
Data Structure	map	Maintain evidence status
String Processing	stringstream	Convert hash values to string
Hashing Method	Custom Hash Algorithm	Verify evidence integrity
Development IDE	Code::Blocks/VS Code	Code writing and execution
Compiler	g++ Compiler	Program compilation
Platform	Console Application	User interaction through menu

8. System Architecture & UI Design

System Architecture

- **User Interface Layer:** Menu-driven console
- **Processing Layer:**
 - Evidence Manager
 - Hash Generator
 - Analysis Module

- **Storage Layer:**
 - Local files (logs & reports)

UI Design (Console Example)

1. Add Evidence File
2. View Evidence List
3. Analyze File
4. Generate Hash
5. View Logs
6. Generate Report
7. Exit

9. Project Timeline & Milestone

Phase	Duration	Task
Planning	Week 1	Requirement analysis
Design	Week 2	Class design & flow
Development	Week 3–5	Core features coding
Testing	Week 6	Debugging
Finalization	Week 7	Documentation & report

10. Expected Deliverables

- C++ source code (.cpp files)
- Executable program
- Project documentation
- Sample evidence dataset
- Output reports (TXT)
- Presentation